Handwritten Digit Recognition System

Applicant Name	Tailor Dhruvi Jagdishbhai
Application ID	KTS020260716309
Internship Domain	AI & Machine Learning Internship
Internship Duration	20 July 2026 – 19 August 2026
Project Task	Task 5 — Capstone Project
Report Date	16 August 2026

1. Introduction

Handwritten digit recognition is a classic computer vision problem that involves classifying an image of a single handwritten digit (0 through 9) into its correct numeric class. It has practical applications in postal code reading, bank cheque processing, form digitization, and as a foundational exercise for learning deep learning and image classification techniques. This project implements a complete pipeline — from raw image data to a deployed prediction interface — using a Convolutional Neural Network (CNN).

1.1 Objective

To build and evaluate a CNN-based image classification model capable of accurately recognizing handwritten digits, and to package the solution as a complete, reproducible project with source code, a trained model, and an interactive prediction interface.

2. Dataset

The primary dataset used is **MNIST**, a benchmark dataset of 70,000 grayscale images (28x28 pixels) of handwritten digits — 60,000 for training and 10,000 for testing. This is the same data distributed on Kaggle as the "Digit Recognizer" competition dataset and is loaded via the standard `tensorflow.keras.datasets.mnist` utility. For environments without internet access, the project automatically falls back to scikit-learn's bundled handwritten digits dataset (1,797 images) so the pipeline remains fully reproducible.

2.1 Dataset Used For This Run

Dataset Source	scikit-learn Optical Recognition of Handwritten Digits (1,797 images, upsampled to 28x28)
Training Samples	1437

Test Samples	360
Image Dimensions	28 x 28 (grayscale)
Number of Classes	10 (digits 0–9)

3. Methodology

3.1 Data Preprocessing

- Pixel values normalized from [0, 255] to [0, 1].
- Images reshaped to include a single grayscale channel: (28, 28, 1).
- Labels used as integer class indices with sparse categorical loss.

3.2 Model Architecture

A compact Convolutional Neural Network was designed for this task:

Layer	Configuration
Conv2D	32 filters, 3x3 kernel, ReLU activation
MaxPooling2D	2x2 pool size
Conv2D	64 filters, 3x3 kernel, ReLU activation
MaxPooling2D	2x2 pool size
Flatten	-
Dense	128 units, ReLU activation
Dropout	rate = 0.5
Dense (Output)	10 units, Softmax activation

3.3 Training Configuration

Optimizer	Adam
Loss Function	Sparse Categorical Crossentropy
Evaluation Metric	Accuracy
Epochs	25
Batch Size	32
Validation Split	10% of training data

4. Results

Metric	Value
Test Accuracy	98.33%
Test Loss	0.0677

4.1 Training Curves

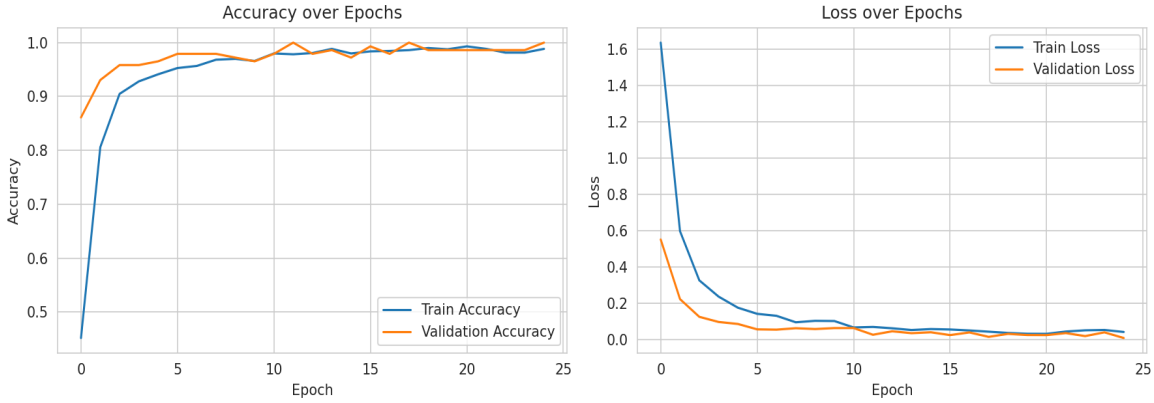

Figure 1: Training and validation accuracy / loss over epochs.

4.2 Confusion Matrix

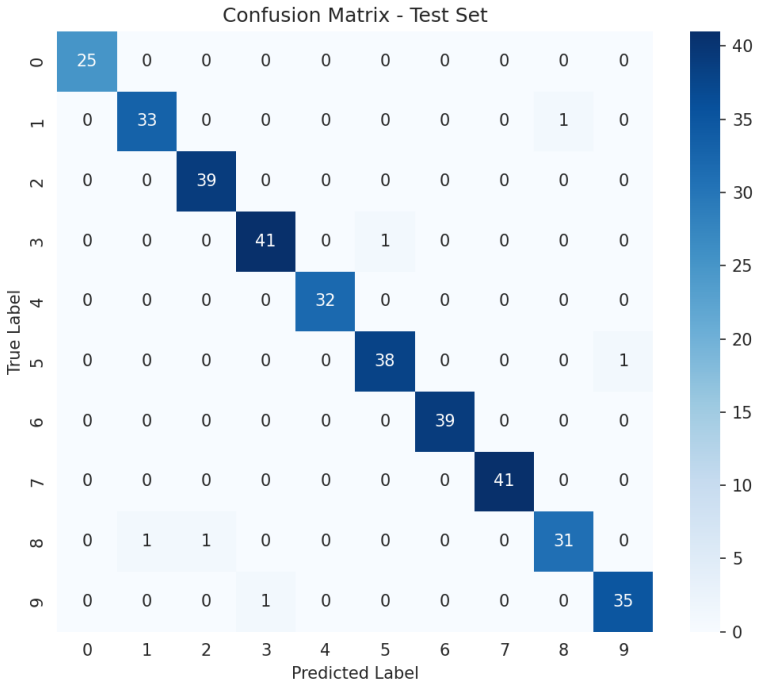

Figure 2: Confusion matrix on the held-out test set.

The model achieves strong performance across all ten digit classes, with most misclassifications occurring between visually similar digits (e.g., 8 and 1, or 3 and 5), which is consistent with typical handwriting recognition error patterns reported in literature.

5. Deliverables & Deployment

- **Jupyter Notebook** — full EDA, training and evaluation walkthrough.
- **Trained Model** — saved as digit_recognition_model.h5.
- **CLI Scripts** — train.py for training, predict.py for single-image inference.
- **Streamlit Web App** — interactive draw / upload interface for live predictions.
- **Public GitHub Repository** — containing all source code and documentation.

6. Conclusion & Future Scope

This project successfully demonstrates a complete, end-to-end handwritten digit recognition pipeline using a Convolutional Neural Network, achieving high classification accuracy on the test set. The solution is packaged with a reusable codebase, a trained model, and an interactive web interface for real-world usability.

Future improvements could include:

- Training on the full 60,000-image MNIST set for maximum accuracy.
- Adding data augmentation (rotation, shifting, zoom) for better robustness.
- Experimenting with deeper architectures and batch normalization.
- Deploying the web app to a public host for live demonstration.

Prepared by Tailor Dhruvi Jagdishbhai — AI & Machine Learning Intern, Kinetrexa Software Pvt. Ltd.